

Hypervisor-PTW

MicroArch Specification v0.1

April 9, 2022

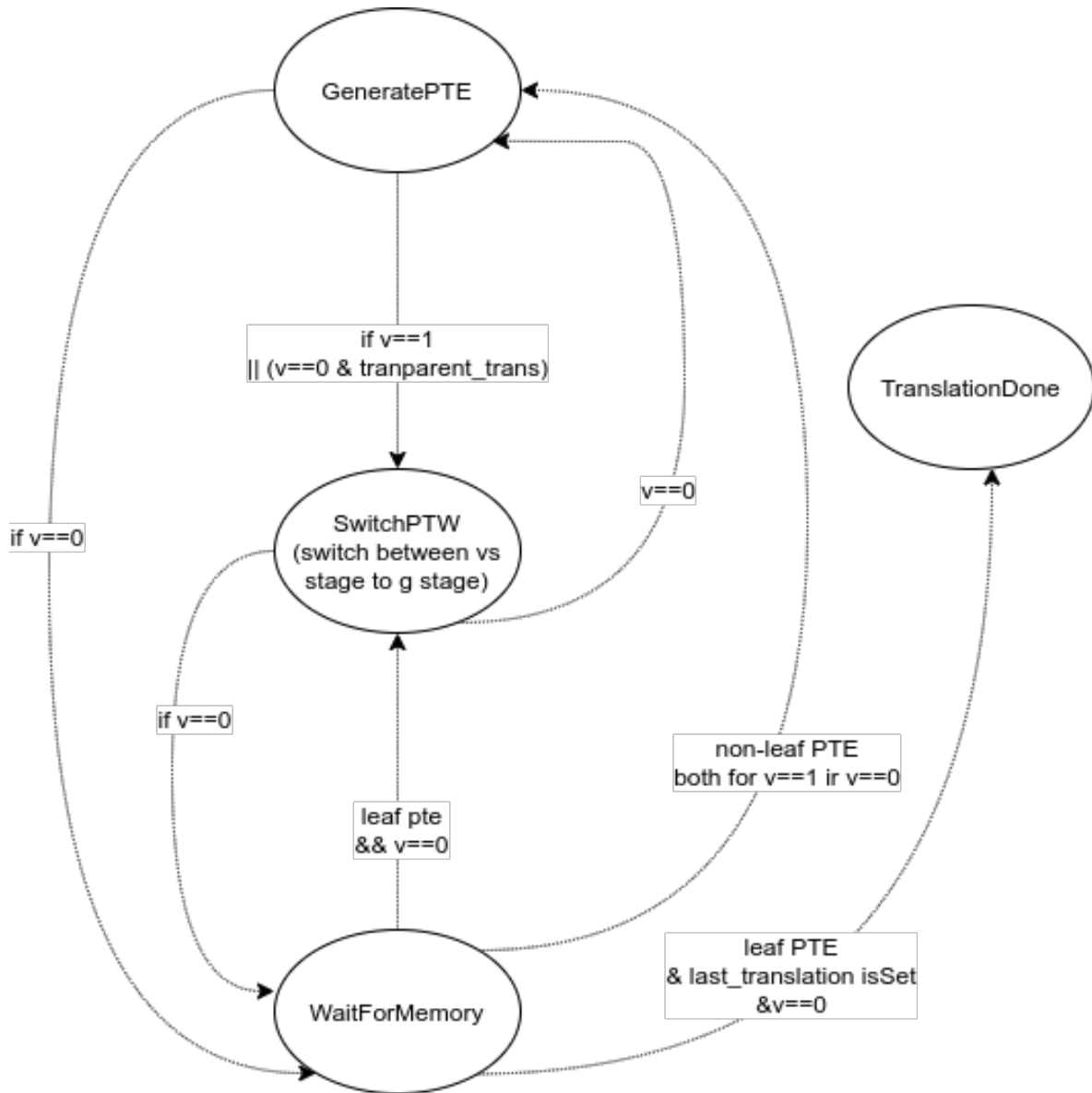
Abstract

The PTW module is responsible for translating the virtual address to physical address, the above block diagram represents PTW module working for hypervisor. Hypervisor supports two-stage address translation, for any virtual memory access the original virtual address is converted in the first stage byu VS-level address translation, as controlled by vsatp register, into guest physical address. The guest physical address is then converted in the second stage by guest physical address translation as controlled by the hgap register into a supervisor physical address, the two stage translation are known as VS-stage and G-stage translation

Contents

1 State Diagram	2
2 Working inside States	3
3 Block Diagram Explanation	7
4 Interface Explanation	8
5 Data Structure used description	9

1 State Diagram



2 Working inside States

GeneratePTE (for request with V=1)

Register used

ff.request_frm_core: the request to ptw module is enqueued here.

rg_level_VS :rg_level_VS : tell which level the VS-stage translation is, this gets sets when request is receive in the subifc_request_frm_core

rg_a_VS : this is the same register used in the address translation algo description as ‘a’. It is used to store the ppn value albeit the root ppn from the atp.ppn or the intermediate ppn generated during translation by the pte.ppn

- Request received
 - Fields inside request
 - * VA : here for V=1 the VA is the **GVA**
 - * ATP: here for V=1 the ATP is **VSSATP**
 - * V: V bit representing which stage of translation this is V==1 means that its VS-stage translation
- If the translation is a transparent translation then GPA is GVA and save that in **rg_gpa** and set the **last_translation** flag
- If it is not a transparent translation then Generate address of PTE from the virtual address and the ppn in the **rg_a_VS** its value is either the ATP.ppn or the intermediate ppn generated during translation by the pte.ppn
 - If the **rg_level_VS** has the max value which is checked by seeing the mode VS is working on and setting the max value, the atp.ppn value and the va[rg_level_VS] are concatenated to make the address of PTE
 - If the **rg_level_VS** is less than the max value then **rg_a_VS** and va[rg_level_VS] is used to generate the address of PTE.
- The generated address is the address for the pte, this is the physical address for the VS or c/a **GPA** This GPA need to get translated to host physical address to get the data pointed out by the address GPA the host physical address here is termed as HPA
- Save the GPA to a register **rg_gpa**.
- To get a translation of GPA to HPA we need to switch the ptw to **G-stage translation** so for a transition of ptw change the state to **SwitchPTW**.

SwitchPTW (for request with V=1)

- Check the V bit in the ff_req_queue, to see which stage of translation the request came from if request came from VS-stage translation then we change the PTW to G-stage translation and vice versa
- Save the request inside the ff_req_queue to ff_req_of_tran_gva_to_hpa, and dequeue the request in ff_req_queue. This was to done so we can have the main request given to PTW by the TLB miss.
- Create a new request with V=0 and enqueue that to ff_req_que.
 - Fields inside request
 - * VA : here we have set V=0 so its a G-stage translation so va is GPA which we get from rg.GPA

- * ATP: here for V=0 the ATP is HGATP
- * V: V bit representing which stage of translation this is V==0 meaning G-stage translation
- Also set the rg_levels_G register using the HGATP.mode value
- Change the state to generate_pte

GeneratePTE (for request with V=0)

Register used

ff_request_frm_core: the request to ptw module is enqueued here.

rg_level_G : tell which level the G-stage translation is, this gets sets when request is receive in the subifc_request_frm_core

rg_a_G : this is the same register used in the address translation algo description as 'a'. It is used to store the ppn value albeit the root ppn from the atp.ppn or the intermediate ppn generated during translation by the pte.ppn

- Request received
 - Fields inside request
 - * VA : here for V=0 the VA is the GPA
 - * ATP: here for V=0 the ATP is HGATP
 - * V: V bit representing which stage of translation this is V==0 means that its G-stage translation
- If the translation is a transparent translation/Bare metal translation which can be checked by the HGATP.mode == 0 then copy the value of GPA to rg_hpa and since we got the HPA corresponding to GPA we switch the PTW to VS- stage , so to do this change the state to SwtichPTW
- Else if it is not a transparent generate address of PTE from the virtual address and the ppn in the rg_a_G its value is either the ATP.ppn or the intermediate ppn generated during translation by the pte.ppn
 - If the rg_level_G has the max value which is checked by seeing the mode HS is working on and setting the max value, it is atp.ppn value and the va[rg_level_G] is which are concatenated to make the address of PTE
 - If the rg_level_G is less than the max value then rg_a_G and va[rg_level_G] is used to generate the address of PTE.
- The generated address is the address for the pte, this is the physical address for the HS or c/a HPA
- Now we want to see which PTE the HPA points to so, we enqueue the created address to ff_memory_req with other metadata
- And finally change the state to WaitForMemory

WaitForMemory (for request with V=0)

- Check the V bit of the request
- Get the memory value that is the PTE and dequeue the ff
- Do permission checks on the PTE

- FAULTS and TRAP
- Check if the PTE is leaf PTE or not
 - If non leaf PTE then set the `pte.ppn` to `rg.a_G` and decrement the `rg_levels_G` value by one and change the state to `GeneratePTE`
 - If it is a leaf PTE then we get the HPA by concatenating the leaf page `pte.ppn` with the virtual address of the request i.e. GPA here based on the `rg_levels_G` value
 - * If it is leaf PTE and `last_translation` flag is set then set the `translation_done` flag, save the HPA to `rg_hpa` and change the state to `TranslationDone`
- If we get the HPA we got the translation of GPA to HPA in second stage translation so now we will do a lookup for the PTE in VS Page Table which was earlier pointed out by GPA and since we cannot lookup GPA in our host machine we got the translation of that address GPA to HPA.
- We save the HPA inside a reg `rg_hpa`.
- So to switch the PTW from Gstage translation to VS stage we change the state to `SwitchPTW`.

SwitchPTW (for request with V=0)

- Check the V bit in the `ff_req_queue`
- Get the saved request inside `ff_req_of_tran_gva_to_hpa` and enqueue it into `ff_req_queue`
- Created request contain the old request given by the TLB
 - Fields inside request
 - * VA : here for V=1 the VA is the GVA
 - * ATP: here for V=1 the ATP is VSSATP
 - * V: V bit representing which stage of translation this is V==1 means that its VS-stage translation
- Now we want to see which PTE the HPA points to so, we enqueue the `rg_hpa` address to `ff_memory_req` with other metadata
- Change the state to `WaitForMemory`.

WaitForMemory (for request with V=1)

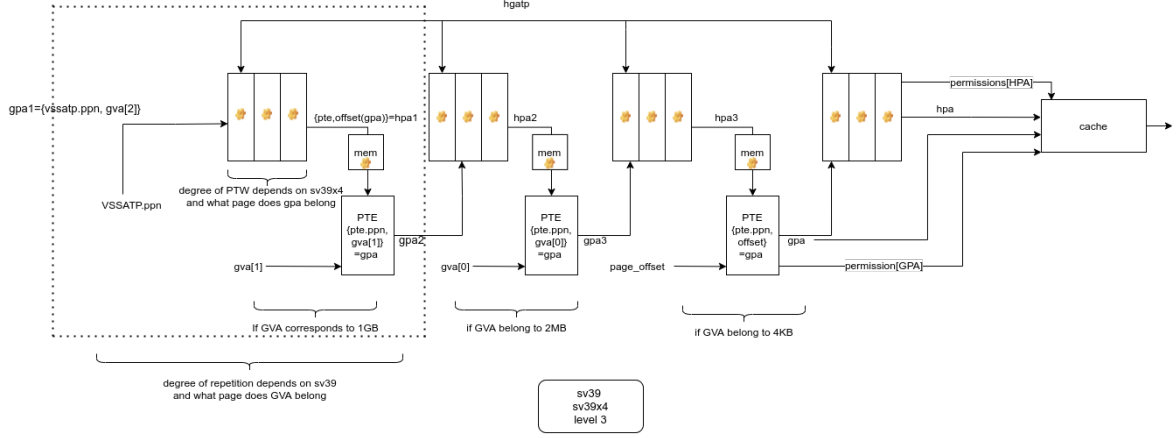
- Check the V bit of the request
- Get the memory value that is the PTE and dequeue the `ff`
- Do permission checks on the PTE
- FAULTS and TRAP
- Check if the PTE is leaf PTE or not
 - If non leaf PTE then set the `pte.ppn` to `rg.a_VS` and decrement the `rg_levels_VS` value by one and change the state to `GeneratePTE`
 - If it is a leaf PTE then we get the final GPA by concatenating the leaf page `pte.ppn` with the virtual address of the request i.e. GVA here based on the `rg_levels_VS` value and we set flag `last_translation`
- Change the state to `SwitchPTE`

Translation_done

- Dequeue the ff_req_queue
- Send the response to tlb with GVA, HPA, GPA, HPA[p] GPA[p]
 - GVA we can get from ff_gva_to_hpa_translation
 - HVA we get from rg_hpa
 - GPA we get from rg_gp

3 Block Diagram Explanation

Below is the description for the block diagram for PTW-hypervisor.



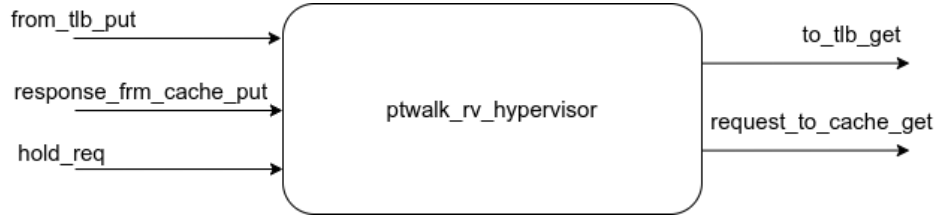
Note for a n-level ptw, n is bounded by two things first one is the virtualization mode the ptw is happening which bounds the max value of n and the page that virtual address belongs to which tell at which level the ptw will get the ppn from for example for a sv39 mode where the virtual address belongs to 2mib page then the ptw will get the ppn from 2-level ptw

Description below assumes that the GVA and GPA belong to 4KiB page, and the VS mode is sv39 and HS mode is sv39x4 The virtual address generated by the core in VS mode need to get translated to physical memory

1. Initially, the core is in vs mode so the virtual address GVA will first get translated to GPA. the value of GPA is the concatenation of vssatp.ppn and gva[2].
2. Gpa is then translated to hpa usig h gatp csr, by a 3-level page table walk. This PTW is the G-stage translation of gpa to hpa
3. The final hpa that is the translation of gpa is created by concatenating the pte.ppn of last level ptw ,in gstage translation and offset which we get from the 12 LSB of GPA
4. This HPA is a physical address of GPA in the host machine so a memory access is done using this address to figure out the PTE the GPA points to.
5. The response of memory access is a pte and the ppn is concatenated with gva[1] to get the gpa2.
6. The above step from step 2 to 5 are 1st level ptw of Vs-stage translation, so the steps 2-5 are repeated 2 more times with appropriate gpa and gva values to for a 3 level PTW in vs mode of translating the gva to gpa, after we get the final gpa a final G-stage translation is done to convert the gpa to hpa and appropriate values are send to response of the ptw which mainly consist of requested GVA whose translatio nis requested, then the GPA along with its permissions bits which are used for the final g-stage translation and hpa along with its permission bits that we get at end of whole ptw

4 Interface Explanation

Below is the description for the block diagram for PTW-hypervisor.



- **from_tlb_put:** Put interface, i.e input port. This interface is used to get the incoming request from TLBs
- **to_tlb_get:** Get interface, i.e output port. This interface is used to send the response of ptwalk to TLBs
- **response_frm_cache_put:** Put interface, i.e. input port. This interface is used to get response of the memory request that is send by the PTWALK module to cache
- **hold_req :** Put interface, i.e. input port. When the core requests a memory it sends out a address to Dcache, Dcache enqueues the request to ptwalk as well as TLB hold_req is used to input the request for which there is a TLB miss so that after the ptwalk is completed ptwalk can enqueue the request to cache and cache can replay the memory request.
- **request_to_cache_get:** Get interface, i.e output port. This sends out the request to cache.

5 Data Structure used description

To be included after 1st iteration of code.....